

Assignment 6 Numerical Integration

Shuifan Wang shepherd.wang@foxmail.com

April 24, 2025

Problem 1

$$(1) \int_0^1 \frac{\sin x}{\sqrt{1-x^2}} dx = 0.89324$$
$$(2) \iint_{x^2+y^2 \leq 2y} (1+2x+y+xy) dx dy = 6.2832$$

```
1 %-----Problem 1-----
2 diary('output1.txt');
3 diary on;
4 clear; clc; close all;
5
6 f1 = @(x) sin(x) ./ sqrt(1 - x.^2);
7 I1 = integral(f1, 0, 1);
8
9 f2 = @(x, y) 1 + 2*x + y + x.*y;
10 lb = @(x) 1 - sqrt(1 - x.^2);
11 ub = @(x) 1 + sqrt(1 - x.^2);
12 I2 = integral2(f2, -1, 1, lb, ub);
13
14 disp(['result1: ', num2str(I1)]);
15 disp(['result2: ', num2str(I2)]);
16
17 diary off;
18
19 % Output:
20 % result1: 0.89324
21 % result2: 6.2832
```

Problem 2

We use Golub–Welsch method to solve the weights and nodes. First we compute the Jacobi matrix

$$\begin{bmatrix} \beta_0 & \alpha_0 & 0 & \cdots & 0 \\ \alpha_0 & \beta_1 & \alpha_1 & \cdots & 0 \\ 0 & \alpha_1 & \beta_2 & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \alpha_{n-1} \\ 0 & \cdots & \cdots & \alpha_{n-1} & \beta_n \end{bmatrix}$$

where $\beta_k = 0$ and $\alpha_k = \frac{k}{(2k-1)(2k+1)}$. Then we calculate all the nodes x_i and weights v_i

$$\mathbf{J}v_i = x_i v_i$$

The result of 4-point Gauss–Legendre is 2.9245409391. Compositing with 20 subintervals and then using affine transform, the result is 2.9253034918, which is exactly the same as what Matlab built-in numerical integral indicates.

```

1  %-----Problem 2-----
2  diary('output2.txt');
3  diary on;
4  clear; clc; close all;
5
6  %% -GaussLegendre Integration
7  n = 4;
8  % Using -GolubWelsch Algorithm
9  [j, i] = meshgrid(0:n-1, 0:n-1);
10 alpha = zeros(n, 1);           % Diagonal ...
    Entries
11 beta = (1:n-1) ./ sqrt(4 * (1:n-1).^2 - 1); % Subdiagonal ...
    Entries
12 J = diag(alpha) + diag(beta, 1) + diag(beta, -1); % Jacobi Matrix
13 [V,D] = eig(J);                % Eigen ...
    Decomposition
14 [x_GL, idx] = sort(diag(D));    % Sorted Nodes
15 w_GL = V(1, idx).^2 * 2;       % Normalized ...
    Weights
16
17 I_GL4 = sum(w_GL' .* exp(x_GL.^2));
18 fprintf('4-point -GaussLegendre on [-1,1]:%.10f\n', I_GL4);
19
20 %% Composite -GaussLegendre
21 m = 20;           % Number of Subintervals
22 a = -1;
23 b = 1;
24 h = (b - a)/m;
25 I_comp = 0;
26 for k = 1:m
27     ak = a + (k-1) * h;  bk = a + k * h;
28     % Affine Transform
29     xk = ((bk - ak)/2) * x_GL + (ak + bk)/2;

```

```

30     wk = w_GL * ((bk - ak)/2);
31     I_comp = I_comp + sum(wk' .* exp(xk.^2));
32 end
33 fprintf('Composite GL :%.10f\n', I_comp);
34
35 %% Compare with Exact Integral
36 I_exact = integral(@(x) exp(x.^2), -1, 1, 'AbsTol', 1e-14);
37 fprintf('MATLAB integral (exact):%.10f\n', I_exact);
38
39 diary off;
40
41 % Output:
42 % 4-point -GaussLegendre on [-1,1]:2.9245409391
43 % Composite GL :2.9253034918
44 % MATLAB integral (exact):2.9253034918

```

Problem 3

$$\int_0^{+\infty} \frac{e^{-x}}{1+x^4} dx \approx 0.598326866854 (= 0.630477834918)$$

The method is the same as problem 2, with $\beta_k = 2k + 1$ and $\alpha_k = k$. We can verify that algebraic precision of degree 11 is satisfied, since the error is within 10^{-7} .

```

1  %-----Problem 3-----
2  diary('output3.txt');
3  diary on;
4  clear; clc; close all;
5
6  %% -GaussLegendre Integration
7  n = 6;
8  % Using -GolubWelsch Algorithm
9  i = (0:n-1)';
10 alpha = 2*i + 1;
11 beta = (1:n-1)';
12 % Jacobi Matrix
13 J = diag(alpha) + diag(beta, 1) + diag(beta, -1);
14 % Nodes and Weights
15 [V,D] = eig(J);
16 [x_GLag, idx] = sort(diag(D));
17 w_GLag = V(1, idx).^2;
18
19 %% Evaluate the Integral and Compare
20 f = @(x) 1./(1 + x.^4);
21 I_Lag6 = sum(w_GLag' .* f(x_GLag));
22 fprintf('6-point -GaussLaguerre (e^{-x}/(1+x^4)) dx = %.12f\n', ...
        I_Lag6);
23

```

```

24 I_exact = integral(@(x) exp(-x) ./ (1 + x.^4), 0, Inf, 'AbsTol', ...
    1e-12);
25 fprintf('MATLAB integral (exact): %.12f\n', I_exact);
26
27 %% Testing the Accuracy
28 max_k = 12;
29
30 for k = 0:max_k
31     f = @(x) x.^k;
32     I_Lag = sum(w_GLag' .* f(x_GLag));
33
34     if k == 0
35         I_exact = 1;
36     else
37         I_exact = factorial(k);
38     end
39
40     error = abs(I_Lag - I_exact);
41     fprintf('k=%2d: I_Lag = %.12e, I_exact = %.12e, error = ...
        %.12e\n', ...
        k, I_Lag, I_exact, error);
42
43 end
44
45 diary off;
46
47 % Output:
48 % 6-point -GaussLaguerre (e^{-x}/(1+x^4)) dx = 0.598326866854
49 % MATLAB integral (exact): 0.630477834918
50 % k= 0: I_Lag = 1.000000000000e+00, I_exact = 1.000000000000e+00, ...
    error = 4.440892098501e-16
51 % k= 1: I_Lag = 1.000000000000e+00, I_exact = 1.000000000000e+00, ...
    error = 2.220446049250e-16
52 % k= 2: I_Lag = 2.000000000000e+00, I_exact = 2.000000000000e+00, ...
    error = 4.440892098501e-16
53 % k= 3: I_Lag = 6.000000000000e+00, I_exact = 6.000000000000e+00, ...
    error = 8.881784197001e-16
54 % k= 4: I_Lag = 2.400000000000e+01, I_exact = 2.400000000000e+01, ...
    error = 7.105427357601e-15
55 % k= 5: I_Lag = 1.200000000000e+02, I_exact = 1.200000000000e+02, ...
    error = 0.000000000000e+00
56 % k= 6: I_Lag = 7.200000000000e+02, I_exact = 7.200000000000e+02, ...
    error = 0.000000000000e+00
57 % k= 7: I_Lag = 5.040000000000e+03, I_exact = 5.040000000000e+03, ...
    error = 9.094947017729e-13
58 % k= 8: I_Lag = 4.032000000000e+04, I_exact = 4.032000000000e+04, ...
    error = 2.910383045673e-11
59 % k= 9: I_Lag = 3.628800000000e+05, I_exact = 3.628800000000e+05, ...
    error = 4.074536263943e-10
60 % k=10: I_Lag = 3.628800000000e+06, I_exact = 3.628800000000e+06, ...
    error = 4.656612873077e-09
61 % k=11: I_Lag = 3.991680000000e+07, I_exact = 3.991680000000e+07, ...
    error = 5.215406417847e-08
62 % k=12: I_Lag = 4.784832000000e+08, I_exact = 4.790016000000e+08, ...
    error = 5.184000000007e+05

```

Problem 4

$$\iint_D (x^2 + y^2) \sin(x + y) dx dy = 102.3888755382$$

D is the quadrilateral region enclosed by $A_1(1, 2)$, $A_2(4, 3)$, $A_3(5, 5)$, and $A_4(2, 6)$. We first use bilinear mapping to project $(\xi, \eta) \in [-1, 1]^2$ into (x, y) :

$$x(\xi, \eta) = \sum_{i=1}^4 x_i N_i(\xi, \eta)$$

$$y(\xi, \eta) = \sum_{i=1}^4 y_i N_i(\xi, \eta)$$

where $N_i(\xi, \eta)$ are the basis functions for bilinear interpolation

$$N_1(\xi, \eta) = \frac{1}{4}(1 - \xi)(1 - \eta)$$

$$N_2(\xi, \eta) = \frac{1}{4}(1 + \xi)(1 - \eta)$$

$$N_3(\xi, \eta) = \frac{1}{4}(1 + \xi)(1 + \eta)$$

$$N_4(\xi, \eta) = \frac{1}{4}(1 - \xi)(1 + \eta)$$

Therefore we have

$$\iint_D f(x, y) dx dy = \iint_{[-1, 1]^2} f(x(\xi, \eta), y(\xi, \eta)) |\det J(\xi, \eta)| d\xi d\eta$$

, where

$$J = \begin{bmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial x}{\partial \eta} \\ \frac{\partial y}{\partial \xi} & \frac{\partial y}{\partial \eta} \end{bmatrix}$$

can be calculated through numerical derivatives.

```

1  %-----Problem 4-----
2  diary('output4.txt');
3  diary on;
4  clear; clc; close all;
5
6  %% Initialize and Run
7  X = [1, 4, 5, 2];
8  Y = [2, 3, 5, 6];
9  n = 10;
10 % Compute Gauss-Legendre Nodes and Weights on [-1, 1]
11 [x, w] = lgwt(n, -1, 1);
12 [xi_grid, eta_grid] = meshgrid(x, x);
13 weights = w' * w;
```

```

14 integral_val = 0;
15
16 %% Loop over Gauss points
17 for i = 1:n
18     for j = 1:n
19         xi = xi_grid(i, j); % Current xi
20         eta = eta_grid(i, j); % Current eta
21
22         % Shape Functions for Bilinear Quadrilateral
23         N = [(1 - xi) * (1 - eta)/4, (1 + xi) * (1 - eta)/4, ...
24             (1 + xi) * (1 + eta)/4, (1 - xi) * (1 + eta)/4];
25
26         % Physical Coordinates at Current Gauss Point
27         x_phys = sum(X .* N); % Physical x-Coordinate
28         y_phys = sum(Y .* N); % Physical y-Coordinate
29
30         % Derivatives of Shape Functions
31         dN_dxi = [-(1 - eta)/4, (1 - eta)/4, (1 + eta)/4, -(1 + ...
32             eta)/4];
33         dN_deta = [-(1 - xi)/4, -(1 + xi)/4, (1 + xi)/4, (1 - ...
34             xi)/4];
35
36         % Jacobian Components
37         dx_dxi = sum(X .* dN_dxi);
38         dy_dxi = sum(Y .* dN_dxi);
39         dx_deta = sum(X .* dN_deta);
40         dy_deta = sum(Y .* dN_deta);
41
42         % Jacobian Matrix
43         J = [dx_dxi, dx_deta; dy_dxi, dy_deta];
44         % Determinant of Jacobian
45         detJ = abs(det(J));
46
47         % Integrand Function
48         fxy = (x_phys^2 + y_phys^2) * sin(x_phys + y_phys);
49
50         % Accumulate Integral Value
51         integral_val = integral_val + fxy * weights(i, j) * detJ;
52     end
53 end
54
55 %% Display the Result
56 fprintf('Approximated integral value: %.10f\n', integral_val);
57
58 diary off;
59
60 % lgwt.m
61 % Compute Gauss-Legendre Nodes and Weights
62 function [x, w] = lgwt(n, a, b)
63     % Computes the Legendre-Gauss Nodes x and Weights w on ...
64     % Interval [a, b]
65     i = 1:n-1;
66     beta = 0.5 ./ sqrt(1 - (2*i).^(-2)); % Recurrence ...
67     % Coefficients
68     T = diag(beta, 1) + diag(beta, -1); % Jacobi Matrix
69     [V, D] = eig(T); % Eigenvalue ...
70     % Decomposition

```

```

66     x = diag(D);                                % Nodes in [-1, 1]
67     [x, idx] = sort(x);
68     w = 2 * V(1, idx).^2;                        % Weights
69     % Affine Transformation
70     x = 0.5*(b - a) * x + 0.5*(a + b);
71     w = 0.5*(b - a) * w;
72 end
73
74 % Output:
75 % Approximated integral value: 102.3888755382

```